

Since the number of solutions is required, the entire tree must be searched, even if one solution is found quickly. So it doesn't matter from a time perspective whether DFS or BFS is used. Since DFS takes less space, it is the search of choice for this problem.

### **Udder Travel**

The Udder Travel cow transport company is based at farm A and owns one cow truck which it uses to pick up and deliver cows between seven farms A, B, C, D, E, F, and G. The (commutative) distances between farms are given by an array. Every morning, Udder Travel has to decide, given a set of cow moving orders, the order in which to pick up and deliver cows to minimize the total distance traveled. Here are the rules:

1. The truck always starts from the headquarters at farm A and must return there when the day's deliveries are done.
2. The truck can only carry one cow at time.
3. The orders are given as pairs of letters denoting where a cow is to be picked up followed by where the cow is to be delivered.

Your job is to write a program that, given any set of orders, determines the shortest route that takes care of all the deliveries, while starting and ending at farm A.

Since all possibilities must be tried in order to ensure the best one is found, the entire tree must be searched, which takes the same amount of time whether using DFS or BFS. Since DFS uses much less space and is conceptually easier to implement, use that.

### **Desert Crossing**

A group of desert nomads is working together to try to get one of their group across the desert. Each nomad can carry a certain number of quarts of water, and each nomad drinks a certain amount of water per day, but the nomads can carry differing amounts of water, and require different amounts of water. Given the carrying capacity and drinking requirements of each nomad, find the minimum number of nomads required to get at least one nomad across the desert.

All the nomads must survive, so every nomad that starts out must either turn back at some point, carrying enough water to get back to the start or must reach the other side of the desert. However, if a nomad has surplus water when it is time to turn back, the water can be given to their friends, if their friends can carry it.

This problem actually is two recursive problems: one recursing on the set of nomads to use, the other on when the nomads turn back. Depth-first search with iterative deepening works well here to determine the nomads required, trying first if any one can make it across by themselves, then seeing if two work together to get across, etc.

### Addition Chains

An addition chain is a sequence of integers such that the first number is 1, and every subsequent number is the sum of some two (not necessarily unique) numbers that appear in the list before it. For example, 1 2 3 5 is such a chain, as 2 is  $1+1$ , 3 is  $2+1$ , and 5 is  $2+3$ . Find the minimum length chain that ends with a given number.

Depth-first search with iterative deepening works well here, as DFS has a tendency to first try 1 2 3 4 5 ... n, which is really bad and the queue grows too large very quickly for BFS.

### Informed Search

Unlike Uninformed Search, Informed Search knows some information that can be used to improve the path selection. Examples of Informed Search: Best First Search, Heuristic Search such as A\*.

### Best First Search

we define a function  $f(n) = g(n)$  where  $g(n)$  is the estimated value from node 'n' to goal. This search is "informed" because we do a calculation to estimate  $g(n)$

### A\* Search

$f(n) = h(n) + g(n)$ , similar to Best First Search, it uses  $g(n)$ , but also uses  $h(n)$ , the total cost incurred so far. The best search to consider if you know how to compute  $g(n)$ .

| Components |                                                                                                                                                                                                                                                                                                                                                        |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | <p>Given: a undirected graph (see picture on the right)</p> <p>The <i>component</i> of a graph is a maximal-sized (though not necessarily maximum) subgraph which is connected.</p> <p>Calculate the component of the graph.</p> <p>This graph has three components: <math>\{1,4,8\}</math>, <math>\{2,5,6,7,9\}</math>, &amp; <math>\{3\}</math>.</p> |

### Flood Fill Algorithm

**Flood fill** can be performed three basic ways: depth-first, breadth-first, and breadth-first scanning. The basic idea is to find some node which has not been assigned to a component and to calculate the component which contains. The question is how to calculate the component.

In the depth-first formulation, the algorithm looks at each step through all of the neighbors of the current node, and, for those that have not been assigned to a component yet, assigns them to this component and recurses on them.

In the breadth-first formulation, instead of recursing on the newly assigned nodes, they are added to a queue.

In the breadth-first scanning formulation, every node has two values: component and visited. When calculating the component, the algorithm goes through all of the nodes that have been assigned to that component but not visited yet, and assigns their neighbors to the current component.

The depth-first formulation is the easiest to code and debug, but can require a stack as big as the original graph. For explicit graphs, this is not so bad, but for implicit graphs, such as the problem presented has, the numbers of nodes can be very large.

The breadth-formulation does a little better, as the queue is much more efficient than the run-time stack is, but can still run into the same problem. Both the depth-first and breadth-first formulations run in  $N + M$  time, where  $N$  is the number of vertices and  $M$  is the number of edges.

The breadth-first scanning formulation, however, requires very little extra space. In fact, being a little tricky, it requires no extra space. However, it is slower, requiring up to  $N^2 + M$  time, where  $N$  is the number of vertices in the graph.

### Breadth-First Scanning

```
# component(i) denotes the component that node i is in

1 function flood_fill(new_component)
2 do
3   num_visited = 0
4   for all nodes i
5     if component(i) = -2
6       num_visited = num_visited + 1
7       component(i) = new_component
8       for all neighbors j of node i
9         if component(i) = nil
10          component(i) = -2
11 until num_visited = 0
12 function find_components
13   num_components = 0
14   for all nodes i
15     component(node i) = nil
16   for all nodes i
17     if component(node i) is nil
18       num_components = num_components + 1
19       component(i) = -2
20       flood_fill(component(num_components))
```

Running time of this algorithm is  $O(N^2)$ , where  $N$  is the numbers of nodes. Every edge is traversed twice (once for each end-point), and each node is only marked once.

**Company Ownership**

Given: A weighted directed graph, with weights between 0 and 100.

Some vertex A "owns" another vertex B if:

1.  $A = B$
2. There is an arc from A to B with weight more than 50.
3. There exists some set of vertices  $C_1$  through  $C_k$  such that A owns  $C_1$  through  $C_k$ , and each vertex has an arc of weight  $x_1$  through  $x_k$  to vertex B, and  $x_1 + x_2 + \dots + x_k > 50$ .

Find all (a,b) pairs such that a owns b.

This can be solved via an adaptation of the calculating the vertices reachable from a vertex in a directed graph. To calculate which vertices vertex A owns, keep track of the "ownership percentage" for each node. Initialize them all to zero. Now, at each recursive step, mark the node as owned by vertex A and add the weight of all outgoing arcs to the "ownership percentages." For all percentages that go above 50, recurse into those vertices.

**Street Race**

Given: a directed graph, and a start point and an end point.

Find all points p that any path from the start point to the end must travel through p.

The easiest algorithm is to remove each point in turn, and check to see if the end point is reachable from the start point. This runs in  $O(N(M + N))$  time. Since the original problem stated that  $M \leq 100$ , and  $N \leq 50$ , this will run in time easily.

### Cow Tours

The diameter of a connected graph is defined as the maximum distance between any two nodes of the graph, where the distance between two nodes is defined as the length of the shortest path.

Given a set of points in the plane, and the connections between those points, find the two points which are currently not in the same component, such that the diameter of the resulting component is minimized.

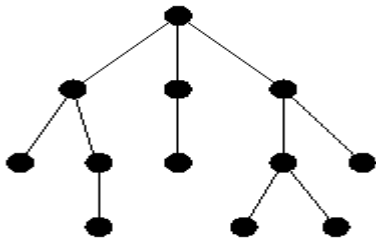
Find the components of the original graph, using the method described above. Then, for each pair of points not in the same component, try placing a connection between them. Find the pair that minimizes the diameter.

### Connected Fields

Farmer John contracted out the building of a new barn. Unfortunately, the builder mixed up the plans of Farmer John's barn with another set of plans. Farmer John's plans called for a barn that only had one room, but the building he got might have many rooms. Given a grid of the layout of the barn, tell Farmer John how many rooms it has.

Analysis: The graph here is on the non-wall grid locations, with edge between adjacent non-wall locations, although the graph should be stored as the grid, and not transformed into some other form, as the grid is so compact and easy to work with.

### Tree



Tree is one of the most efficient data structure used in a computer program. There are many types of tree. Binary tree is a tree that always have two branches, Red-Black-Trees, 2-3-4 Trees, AVL Trees, etc. A well balanced tree can be used to design a good searching algorithm. A Tree is an undirected graph that contains no cycles and is connected. Many trees are what is called rooted, where there is a notion of the "top"

node, which is called the root. Thus, each node has one parent, which is the adjacent node which is closer to the root, and may have any number of children, which are the rest of the nodes adjacent to it. The tree above was drawn as a rooted tree.

## Minimum Spanning Trees

### Spanning trees

A spanning tree of a graph is just a subgraph that contains all the vertices and is a tree. A graph may have many spanning trees; for instance the complete graph on four vertices

```

0---0
|\ /
|X|
|/\
0---0

```

has sixteen spanning trees:

```

0---0  0---0  0  0  0---0
| | |  | | |  |
| | |  | | |  |
| | |  | | |  |
0  0  0---0  0---0  0---0

```

```

0---0  0  0  0  0  0  0
\ /  | \  \ /  \ /
  X  |X   X   X |
 / \  | \  / \  / \
0  0  0  0  0---0  0  0

```

```

0  0  0---0  0  0  0---0
| |  /  | /  \
| \  /  | /  \
| \  /  | /  \
0  0  0---0  0  0  0---0

```

```

0---0  0  0  0  0  0---0
|  /  \ |  /
| \  /  \ |  /
| \  /  \ |  /
0  0  0---0  0---0  0  0

```

### Minimum spanning trees

Now suppose the edges of the graph have weights or lengths. The weight of a tree is just the sum of weights of its edges. Obviously, different trees have different lengths. The problem: how to find the minimum length spanning tree?

### Why minimum spanning trees?

The standard application is to a problem like phone network design. You have a business with several offices; you want to lease phone lines to connect them up with each other; and the phone company charges different amounts of money to connect different pairs of cities. You want a set of lines that connects all your offices with a minimum total cost. It should be a spanning tree, since if a network isn't a tree you can always remove some edges and save money.

A less obvious application is that the minimum spanning tree can be used to approximately solve the traveling salesman problem. A convenient formal way of defining this problem is to find the shortest path that visits each point at least once.

Note that if you have a path visiting all points exactly once, it's a special kind of tree. For instance in the example above, twelve of sixteen spanning trees are actually paths. If you have a path visiting some vertices more than once, you can always drop some edges to get a tree. So in general the MST weight is less than the TSP weight, because it's a minimization over a strictly larger set.

On the other hand, if you draw a path tracing around the minimum spanning tree, you trace each edge twice and visit all points, so the TSP weight is less than twice the MST weight. Therefore this tour is within a factor of two of optimal.

### How to find minimum spanning tree?

The stupid method is to list all spanning trees, and find minimum of list. We already know how to find minima... But there are far too many trees for this to be efficient. It's also not really an algorithm, because you'd still need to know how to list all the trees.

A better idea is to find some key property of the MST that lets us be sure that some edge is part of it, and use this property to build up the MST one edge at a time.

For simplicity, we assume that there is a unique minimum spanning tree. You can get ideas like this to work without this assumption but it becomes harder to state your theorems or write your algorithms precisely.

**Lemma:** Let  $X$  be any subset of the vertices of  $G$ , and let edge  $e$  be the smallest edge connecting  $X$  to  $G-X$ . Then  $e$  is part of the minimum spanning tree.

**Proof:** Suppose you have a tree  $T$  not containing  $e$ ; then we want to show that  $T$  is not the MST. Let  $e=(u,v)$ , with  $u$  in  $X$  and  $v$  not in  $X$ . Then because  $T$  is a spanning tree it contains a unique path from  $u$  to  $v$ , which together with  $e$  forms a cycle in  $G$ . This path has to include another edge  $f$  connecting  $X$  to  $G-X$ .  $T+e-f$  is another spanning tree (it has the same number of edges, and remains connected since you can replace any path containing  $f$  by one going the other way around the cycle). It has smaller weight than  $T$  since  $e$  has smaller weight than  $f$ . So  $T$  was not minimum, which is what we wanted to prove.

### Kruskal's algorithm

We'll start with Kruskal's algorithm, which is easiest to understand and probably the best one for solving problems by hand.

```
Kruskal's algorithm:
sort the edges of  $G$  in increasing order by length
keep a subgraph  $S$  of  $G$ , initially empty
for each edge  $e$  in sorted order
    if the endpoints of  $e$  are disconnected in  $S$ 
        add  $e$  to  $S$ 
return  $S$ 
```

Note that, whenever you add an edge  $(u,v)$ , it's always the smallest connecting the part of  $S$  reachable from  $u$  with the rest of  $G$ , so by the lemma it must be part of the MST.

This algorithm is known as a *greedy algorithm*, because it chooses at each step the cheapest edge to add to  $S$ . You should be very careful when trying to use greedy algorithms to solve other problems, since it usually doesn't work. E.g. if you want to find a shortest path from  $a$  to  $b$ , it might be a bad idea to keep taking the shortest edges. The greedy idea only works in Kruskal's algorithm because of the key property we proved.

**Analysis:** The line testing whether two endpoints are disconnected looks like it should be slow (linear time per iteration, or  $O(mn)$  total). The slowest part turns out to be the sorting step, which takes  $O(m \log n)$  time.

### Prim's algorithm

Rather than build a subgraph one edge at a time, Prim's algorithm builds a tree one vertex at a time.

```

Prim's algorithm:
let T be a single vertex x
while (T has fewer than n vertices) {
    find the smallest edge connecting T to G-T
    add it to T
}

```

Example of Prim's algorithm in C language:

```

/* usedp=>how many points already used
p->array of structures, consisting x,y,& used/not used
this problem is to get the MST of graph with n vertices
which weight of an edge is the distance between 2 points */

usedp=p[0].used=1; /* select arbitrary point as starting point */
while (usedp<n) {
    small=-1.0;

    for (i=0;i<n;i++) if (p[i].used)
        for (j=0;j<n;j++) if (!p[j].used) {
            length=sqrt(pow(p[i].x-p[j].x,2) + pow(p[i].y-p[j].y,2));

            if (small== -1.0 || length<small) {
                small=length;
                smallp=j;
            }
        }
    minLength+=small;

    p[smallp].used=1;
    usedp++ ;
}

```

### Finding Shortest Paths using BFS

This only applicable to graph with unweighted edges, simply do BFS from start node to end node, and stop the search when it encounters the first occurrence of end node.

The **relaxation** process updates the costs of all the vertices, **v**, connected to a vertex, **u**, if we could improve the best estimate of the shortest path to **v** by including (**u,v**) in the path to **v**. The relaxation procedure proceeds as follows:

```

initialize_single_source(Graph G, Node s)
  for each vertex v in Vertices(G)
    G.d[v] := infinity
    G.pi[v] := nil
  G.d[s] := 0;

```

This sets up the graph so that each node has no predecessor (**pi[v] = nil**) and the estimates of the cost (distance) of each node from the source (**d[v]**) are infinite, except for the source node itself (**d[s] = 0**).

Note that we have also introduced a further way to store a graph (or part of a graph - as this structure can only store a spanning tree), the **predecessor sub-graph** - the list of predecessors of each node, **pi[j]**,  $1 \leq j \leq |V|$ . The edges in the predecessor sub-graph are (**pi[v], v**).

The relaxation procedure checks whether the current best estimate of the shortest distance to **v** (**d[v]**) can be improved by going through **u** (*i.e.* by making **u** the predecessor of **v**):

```

relax(Node u, Node v, double w[][])
  if d[v] > d[u] + w[u,v] then
    d[v] := d[u] + w[u,v]
    pi[v] := u

```

## Dijkstra Algorithm

Dijkstra's algorithm (invented by Edsger W. Dijkstra) solves the problem of finding the shortest path from a point in a graph (the *source*) to a destination. It turns out that one can find the shortest paths from a given source to *all* points in a graph in the same time, hence this problem is called the **Single-source shortest paths** problem.

There will also be no cycles as a cycle would define more than one path from the selected vertex to at least one other vertex. For a graph,  $G=(V,E)$  where **V** is a set of vertices and **E** is a set of edges.

Dijkstra's algorithm keeps two sets of vertices: **S** (the set of vertices whose shortest paths from the source have already been determined) *and* **V-S** (the remaining vertices). The other data structures needed are: **d** (array of best estimates of shortest path to each vertex) & **pi** (an array of predecessors for each vertex)

The basic mode of operation is:

1. Initialise d and pi,
2. Set S to empty,

3. While there are still vertices in V-S,
4. Sort the vertices in V-S according to the current best estimate of their distance from source,
5. Add u, the closest vertex in V-S, to S,
6. Relax all the vertices still in V-S connected to u

```

DIJKSTRA(Graph G, Node s)
  initialize_single_source(G, s)
  S := { 0 } /* Make S empty */
  Q := Vertices(G) /* Put the vertices in a PQ */
  while not Empty(Q)
    u := ExtractMin(Q);
    AddNode(S, u); /* Add u to S */
    for each vertex v which is Adjacent with u
      relax(u, v, w)

```

### Bellman-Ford Algorithm

A more generalized single-source shortest paths algorithm which can find the shortest path in a graph with negative weighted edges. If there is no negative cycle in the graph, this algorithm will update each  $d[v]$  with the shortest path from  $s$  to  $v$ , fill up the predecessor list "pi", and return TRUE. However, if there is a negative cycle in the given graph, this algorithm will return FALSE.

```

BELLMAN_FORD(Graph G, double w[][[]], Node s)
  initialize_single_source(G, s)
  for i=1 to |V[G]|-1
    for each edge (u,v) in E[G]
      relax(u, v, w)

  for each edge (u,v) in E[G]
    if d[v] > d[u] + w(u, v) then
      return FALSE
  return TRUE

```

### TEST YOUR BELLMAN FORD KNOWLEDGE

Solve Valladolid Online Judge Problems related with Bellman Ford:

[558 - Wormholes](#), simply check the negative cycle existence.

### Single-source shortest paths in Directed Acyclic Graph (DAG)

There exist a more efficient algorithm for solving Single-source shortest path problem for a Directed Acyclic Graph (DAG). So if you know for sure that your graph is a DAG, you may want to consider this algorithm instead of using Dijkstra.

```
DAG_SHORTEST_PATHS(Graph G, double w[][[]], Node s)
    topologically sort the vertices of G // O(V+E)
    initialize_single_source(G, s)

    for each vertex u taken in topologically sorted order
        for each vertex v which is Adjacent with u
            relax(u, v, w)
```

A sample application of this DAG\_SHORTEST\_PATHS algorithm (as given in CLR book) is to solve critical path problem, i.e. finding the longest path through a DAG, for example: calculating the fastest time to complete a complex task consisting of smaller tasks when you know the time needed to complete each small task and the precedence order of tasks.

### Floyd Warshall

Given a directed graph, the Floyd-Warshall All Pairs Shortest Paths algorithm computes the shortest paths between each pair of nodes in  $O(n^3)$ . In this page, we list down the Floyd Warshall and its variant plus the source codes.

Given:

**w** : edge weights

**d** : distance matrix

**p** : predecessor matrix

$w[i][j]$  = length of direct edge between  $i$  and  $j$

$d[i][j]$  = length of shortest path between  $i$  and  $j$

$p[i][j]$  = on a shortest path from  $i$  to  $j$ ,  $p[i][j]$  is the last node before  $j$ .

#### Initialization

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++) {
        d[i][j] = w[i][j];
        p[i][j] = i;
    }
for (i=0; i<n; i++) d[i][i] = 0;
```

### The Algorithm

```

for (k=0;k<n;k++) /* k -> is the intermediate point */
  for (i=0;i<n;i++) /* start from i */
    for (j=0;j<n;j++) /* reaching j */
      /* if i-->k + k-->j is smaller than the original i-->j */
      if (d[i][k] + d[k][j] < d[i][j]) {

          /* then reduce i-->j distance to the smaller one i->k->j */
          graph[i][j] = graph[i][k]+graph[k][j];
          /* and update the predecessor matrix */
          p[i][j] = p[k][j];
      }

```

In the  $k$ -th iteration of the outer loop, we try to improve the currently known shortest paths by considering  $k$  as an intermediate node. Therefore, after the  $k$ -th iteration we know those shortest paths that only contain intermediate nodes from the set  $\{0, 1, 2, \dots, k\}$ . After all  $n$  iterations we know the real shortest paths.

### Constructing a Shortest Path

```

print_path (int i, int j) {
    if (i!=j)    print_path(i,p[i][j]);
    print(j);
}

```

#### TEST YOUR FLOYD WARSHALL KNOWLEDGE

Solve Valladolid Online Judge Problems related with Floyd Warshall:

[104 - Arbitrage](#) - modify the Floyd Warshall parameter correctly

[423 - MPI Maelstrom](#)

[436 - Arbitrage \(II\)](#) - modify the Floyd Warshall parameter correctly

[567 - Risk](#) - even though you can solve this using brute force

### Transitive Hull

Given a directed graph, the Floyd-Warshall algorithm can compute the Transitive Hull in  $O(n^3)$ . Transitive means, if  $i$  can reach  $k$  and  $k$  can reach  $j$  then  $i$  can reach  $j$ . Transitive Hull means, for all vertices, compute its reachability.

**w** : adjacency matrix

**d** : transitive hull

$w[i][j]$  = edge between  $i$  and  $j$  (0=no edge, 1=edge)  
 $d[i][j]$  = 1 if and only if  $j$  is reachable from  $i$

### Initialization

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        d[i][j] = w[i][j];

for (i=0; i<n; i++)
    d[i][i] = 1;
```

### The Algorithm

```
for (k=0; k<n; k++)
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            /* d[i][j] is true if d[i][j] already true
               or if we can use k as intermediate vertex to reach j from i,
               otherwise, d[i][j] is false */
            d[i][j] = d[i][j] || (d[i][k] && d[k][j]);
```

### TEST YOUR TRANSITIVE HULL FLOYD WARSHALL KNOWLEDGE

Solve Valladolid Online Judge Problems related with Transitive Hull:  
334 - Identifying Concurrent Events - internal part of this problem needs transitive hull, even though this problem is more complex than that.

### MiniMax Distance

Given a directed graph with edge lengths, the Floyd-Warshall algorithm can compute the minimax distance between each pair of nodes in  $O(n^3)$ . For example of a minimax problem, refer to the Valladolid OJ problem below.

**w** : edge weights  
**d** : minimax distance matrix  
**p** : predecessor matrix

$w[i][j]$  = length of direct edge between  $i$  and  $j$   
 $d[i][j]$  = length of minimax path between  $i$  and  $j$

### Initialization

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        d[i][j] = w[i][j];

for (i=0; i<n; i++)
    d[i][i] = 0;
```

### The Algorithm

```
for (k=0; k<n; k++)
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            d[i][j] = min(d[i][j], max(d[i][k], d[k][j]));
```

#### TEST YOUR MINIMAX FLOYD WARSHALL KNOWLEDGE

Solve Valladolid Online Judge Problems related with MiniMax:

[534 - Frogger](#) - select the minimum of longest jumps

[10048 - Audiophobia](#) - select the minimum of maximum decibel along the path

### MaxiMin Distance

You can also compute the maximin distance with the Floyd-Warshall algorithm. Maximin is the reverse of minimax. Again, look at Valladolid OJ problem given below to understand maximin.

**w** : edge weights

**d** : maximin distance matrix

**p** : predecessor matrix

$w[i][j]$  = length of direct edge between  $i$  and  $j$

$d[i][j]$  = length of maximin path between  $i$  and  $j$

### Initialization

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        d[i][j] = w[i][j];

for (i=0; i<n; i++)
    d[i][i] = 0;
```

### The Algorithm

```
for (k=0; k<n; k++)
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            d[i][j] = max(d[i][j], min(d[i][k], d[k][j]));
```

#### TEST YOUR MAXIMIN FLOYD WARSHALL KNOWLEDGE

Solve Valladolid Online Judge Problems related with MaxiMin:

544 - Heavy Cargo - select the maximum of minimal weight allowed along the path.  
10099 - The Tourist Guide - select the maximum of minimum passenger along the path, then divide total passenger with this value to determine how many trips needed.

### Safest Path

Given a directed graph where the edges are labeled with survival probabilities, you can compute the safest path between two nodes (i.e. the path that maximizes the product of probabilities along the path) with Floyd Warshall.

**w** : edge weights

**p** : probability matrix

$w[i][j]$  = survival probability of edge between  $i$  and  $j$

$p[i][j]$  = survival probability of safest path between  $i$  and  $j$

### Initialization

```
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        p[i][j] = w[i][j];

for (i=0; i<n; i++)
    p[i][i] = 1;
```

### The Algorithm

```
for (k=0; k<n; k++)
  for (i=0; i<n; i++)
    for (j=0; j<n; j++)
      p[i][j] = max(p[i][j], p[i][k] * p[k][j]);
```

### Graph Transpose

**Input:** directed graph  $G = (V, E)$

**Output:** graph  $G^T = (V, E^T)$ , where  $E^T = \{(v, u) \text{ in } V \times V : (u, v) \text{ in } E\}$ .  
i.e.  $G^T$  is  $G$  with all its edges reversed.

Describe efficient algorithms for computing  $G^T$  from  $G$ , for both the adjacency-list and adjacency-matrix representations of  $G$ . Analyze the running times of your algorithms.

Using Adjacency List representation, array  $B$  is the new array of Adjacency List  $G^T$

```
for (i=1; i<=p; i++)
  B[i] = nil;

for (i=1; i<=p; i++)
  repeat {
    append i to the end of linked list B[A[i]];
    get next A[i];
  } until A[i] = nil;
```

### Eulerian Cycle & Eulerian Path

#### Euler Cycle

**Input:** Connected, directed graph  $G = (V, E)$

**Output:** A cycle that traverses every edge of  $G$  exactly once, although it may visit a vertex more than once.

Theorem: A directed graph possesses an Eulerian cycle iff

- 1) It is connected
- 2) For all  $\{v\}$  in  $\{V\}$   $\text{indegree}(v) = \text{outdegree}(v)$

#### Euler Path

**Input:** Connected, directed graph  $G = (V, E)$

**Output:** A path from  $v_1$  to  $v_2$ , that traverses every edge of  $G$  exactly once, although it may visit a vertex more than once.

Theorem: A directed graph possesses an Eulerian path iff

- 1) It is connected
- 2) For all  $\{v\}$  in  $\{V\}$   $\text{indegree}(v) = \text{outdegree}(v)$  with the possible exception of two vertices  $v_1, v_2$  in which case,
  - a)  $\text{indegree}(v_1) = \text{outdegree}(v_2) + 1$
  - b)  $\text{indegree}(v_2) = \text{outdegree}(v_1) - 1$

### Topological Sort

**Input:** A directed acyclic graph (DAG)  $G = (V, E)$

**Output:** A linear ordering of all vertices in  $V$  such that if  $G$  contains an edge  $(u, v)$ , then  $u$  appears before  $v$  in the ordering.

If drawn on a diagram, Topological Sort can be seen as a vertices along horizontal line, where all directed edges go from left to right. A directed graph  $G$  is acyclic if and only if a DFS of  $G$  yields no back edge.

#### Topological-Sort( $G$ )

1. call DFS( $G$ ) to compute finishing times  $f[v]$  for each vertex  $v$
2. as each vertex is finished, insert it onto the front of a linked list
3. return the linked list of vertices

Topological-Sort runs in  $O(V+E)$  due to DFS.

### Strongly Connected Components

**Input:** A directed graph  $G = (V, E)$

**Output:** All strongly connected components of  $G$ , where in strongly connected component, all pair of vertices  $u$  and  $v$  in that component, we have  $u \rightsquigarrow v$  and  $v \rightsquigarrow u$ , i.e.  $u$  and  $v$  are reachable from each other.

#### Strongly-Connected-Components( $G$ )

1. call DFS( $G$ ) to compute finishing times  $f[u]$  for each vertex  $u$
2. compute GT, inverting all edges in  $G$  using adjacency list
3. call DFS(GT), but in the main loop of DFS, consider the vertices in order of decreasing  $f[u]$  as computed in step 1
4. output the vertices of each tree in the depth-first forest of step 3 as a separate strongly connected component.

Strongly-Connected-Components runs in  $O(V+E)$

## CHAPTER 13 COMPUTATIONAL GEOMETRY

Computational Geometry is an important subject. Mastering this subject can actually help you in programming contests since every contest usually include 1-2 geometrical problems.<sup>[2]</sup>

### Geometrical objects and its properties

#### Earth coordinate system

People use *latitudes* (horizontal lines) and *longitudes* (vertical lines) in Earth coordinate system.

Longitude spans from 0 degrees (Greenwich) to +180° East and -180° West. Latitude spans from 0 degrees (Equator) to +90° (North pole) and -90° (South pole).

The most interesting question is what is the spherical / geographical distance between two cities p and q on earth with radius r, denoted by (p\_lat, p\_long) to (q\_lat, q\_long). All coordinates are in radians. (i.e. convert [-180..180] range of longitude and [-90..90] range of latitudes to [-pi..pi] respectively).

After deriving the mathematical equations. The answer is as follow:

```
spherical_distance(p_lat, p_long, q_lat, q_long) =
acos( sin(p_lat) * sin(q_lat) + cos(p_lat) * cos(q_lat) * cos(p_long -
q_long) ) * r

since cos(a-b) = cos(a)*cos(b) + sin(a)*sin(b), we can simplify the above
formula to:

spherical_distance(p_lat, p_long, q_lat, q_long) =
acos( sin(p_lat) * sin(q_lat) +
      cos(p_lat) * cos(q_lat) * cos(p_long) * cos(q_long) +
      cos(p_lat) * cos(q_lat) * sin(p_long) * sin(q_long)
    ) * r
```

#### TEST YOUR EARTH COORDINATE SYSTEM KNOWLEDGE

Solve UVa problems related with Earth Coordinate System:

[535 - Globetrotter](#)

[10075 - Airlines](#) - combined with all-pairs shortest path

**Convex Hull**

Basically, Convex Hull is the most basic and most popular computational geometry problem. Many algorithms are available to solve this efficiently, with the best lower bound  $O(n \log n)$ . This lower bound is already proven.

**Convex Hull problem (2-D version):**

**Input:** A set of points in Euclidian plane

**Output:** Find the minimum set of points that enclosed all other points.

**Convex Hull algorithms:**

- a. Jarvis March / Gift Wrapping
- b. Graham Scan
- c. Quick Hull
- d. Divide and Conquer

## CHAPTER 14 VALLADOLID OJ PROBLEM CATEGORY<sup>[2]</sup>

| Math                                   |                                                                                                                                                                                                                           |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (General)                              | 113,202,256,275,276,294326,332,347,350,356,374,377,382,386,412,465,471,474,485,498550,557,568,594,725,727,846,10006,10014,10019,10042,10060,10071,10093,10104,10106,10107,10110,10125,10127,10162,10190,10193,10195,10469 |
| Prime Numbers                          | 406,516,543,583,686,10140,10200,10490                                                                                                                                                                                     |
| Geometry                               | 190,191,378,438,476,477,478,10112,10221,10242,10245,10301,10432,10451                                                                                                                                                     |
| Big Numbers                            | 324,424,495,623,713,748,10013,10035,10106,10220,10334                                                                                                                                                                     |
| Base Numbers                           | 343,355,389,446,575,10183,10551                                                                                                                                                                                           |
| Combinations / Permutations            | 369,530                                                                                                                                                                                                                   |
| Theories / Formulas                    | 106,264,486,580                                                                                                                                                                                                           |
| Factorial                              | 160,324,10323,10338                                                                                                                                                                                                       |
| Fibonacci                              | 495,10183,10334,10450                                                                                                                                                                                                     |
| Sequences                              | 138,10408                                                                                                                                                                                                                 |
| Modulo                                 | 10176,10551                                                                                                                                                                                                               |
| Dynamic Programming                    |                                                                                                                                                                                                                           |
| General                                | 108,116,136,348,495,507,585,640,836,10003,10036,10074,10130,10404                                                                                                                                                         |
| Longest Inc/Decreasing Subsequence     | 111,231,497,10051,10131                                                                                                                                                                                                   |
| Longest Common Subsequence             | 531,10066,10100,10192,10405                                                                                                                                                                                               |
| Counting Change                        | 147,357,674                                                                                                                                                                                                               |
| Edit Distance                          | 164,526                                                                                                                                                                                                                   |
| Graph                                  |                                                                                                                                                                                                                           |
| Floyd Warshall All-Pairs Shortest Path | 1043,436,534,544,567,10048,10099,10171,112,117,122,193,336,352,383,429,469,532,536,590,614,615,657,677,679,762,785,10000,10004,10009,10010,10116,10543                                                                    |
| Network Flow                           | 820,10092,10249                                                                                                                                                                                                           |
| Max Bipartite Matching                 | 670,753,10080                                                                                                                                                                                                             |
| Flood Fill                             | 352,572                                                                                                                                                                                                                   |
| Articulation Point                     | 315,796                                                                                                                                                                                                                   |
| MST                                    | 10034,10147,10397                                                                                                                                                                                                         |
| Union Find                             | 459,793,10507                                                                                                                                                                                                             |
| Chess                                  | 167,278,439,750                                                                                                                                                                                                           |
| Mixed Problems                         |                                                                                                                                                                                                                           |
| Anagram                                | 153,156,195,454,630                                                                                                                                                                                                       |
| Sorting                                | 120,10152,10194,10258                                                                                                                                                                                                     |
| Encryption                             | 458,554,740,10008,10062                                                                                                                                                                                                   |
| Greedy Algorithm                       | 10020,10249,10340                                                                                                                                                                                                         |
| Card Game                              | 162,462,555                                                                                                                                                                                                               |
| BNF Parser                             | 464,533                                                                                                                                                                                                                   |
| Simulation                             | 130,133,144,151,305,327,339,362,379402,440,556,637,758,10033,10500                                                                                                                                                        |
| Output-related                         | 312,320,330,337,381,391,392400,403,445,488,706,10082                                                                                                                                                                      |

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ad Hoc</b>             | 101,102,103, 105,118, 119,121,128, 142,145,146,154, 155,187,195220,227,232, 271,272,291,297,299,300,311,325,333,335,340,344,349,353,380,384,394,401,408,409,413,414,417,434,441,442,444,447,455,457,460,468,482,483,484,489,492,494,496,499537,541,542,551,562,573,574,576,579,586,587,591602,612,616,617,620,621,642,654,656,661,668,671,673729,755,837,10015,10017,10018,10019,10025,10038, 10041,10045,10050,10055,10070,10079,10098,10102, 10126, 10161,10182,10189, 10281,10293, 10487 |
| <b>Array Manipulation</b> | 466,10324,10360,10443                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Binary Search</b>      | 10282,10295,10474                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Backtracking</b>       | 216,291422,524,529, 539, 571, 572, 574,10067,10276,10285,10301,10344,10400,10422,10452                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>3n+1 Problem</b>       | 100,371,694                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

This problems are available at (<http://acm.uva.es/p>). New problems are added after each online contest at Valladolid Online Judge as well as after each ACM Regional Programming Contest, problems are added to live ACM archive (<http://cii-judge.baylor.edu/>).

## **APPENDIX A**

### **ACM PROGRAMMING PROBLEMS**

---



This part of this book contains some interesting problems from ACM/ICPC. Problems are collected from Valladolid Online Judge. You can see the reference section for finding more problem sources.

## Find the ways !

### The Problem

An American, a Frenchman and an Englishwoman had been to Dhaka, the capital of Bangladesh. They went sight-seeing in a taxi. The three tourists were talking about the sites in the city. The American was very proud of tall buildings in New York. He boasted to his friends, "Do you know that the Empire State Building was built in three months?"

"Really?" replied the Frenchman. "The Eiffel Tower in Paris was built in only one month! (However, The truth is, the construction of the Tower began in January 1887. Forty Engineers and designers under Eiffel's direction worked for two years. The tower was completed in March 1889.)

"How interesting!" said the Englishwoman. "Buckingham Palace in London was built in only two weeks!!"

At that moment the taxi passed a big slum ( However, in Bangladesh we call it "Bostii"). "What was that? When it was built ?" The Englishwomen asked the driver who was a Bangladeshi.

"I don't know!" , answered the driver. "It wasn't there yesterday!"

However in Bangladesh, illegal establishment of slums is a big time problem. Government is trying to destroy these slums and remove the peoples living there to a far place, formally in a planned village outside the city. But they can't find any ways, how to destroy all these slums!

Now, can you imagine yourself as a slum destroyer? In how many ways you can destroy  $k$  slums out of  $n$  slums ! Suppose there are 10 slums and you are given the permission of destroying 5 slums, surly you can do it in 252 ways, which is only a 3 digit number, Your task is to find out the digits in ways you can destroy the slums !

### The Input

The input file will contain one or more test cases. Each test case consists of one line containing two integers  $n$  ( $n \geq 1$ ) and  $k$  ( $1 \leq k \leq n$ ).

**Sample Input**

```
20 5
100 10
200 15
```

**Sample Output**

```
5
14
23
```

---

*Problem Setter : Ahmed Shamsul Arefin*

## I Love Big Numbers !

**The Problem**

A Japanese young girl went to a Science Fair at Tokyo. There she met with a Robot named Mico-12, which had AI (You must know about AI-Artificial Intelligence). The Japanese girl thought, she can do some fun with that Robot. She asked her, "Do you have any idea about maths ?". "Yes! I love mathematics", The Robot replied.

"Okey ! Then I am giving you a number, you have to find out the Factorial of that number. Then find the sum of the digits of your result!. Suppose the number is 5. You first calculate  $5!=120$ , then find sum of the digits  $1+2+0=3$ . Can you do it?"

"Yes. I can do!" Robot replied. "Suppose the number is 100, what will be the result ?". At this point the Robot started thinking and calculating. After a few minutes the Robot head burned out and it cried out loudly "Time Limit Exceeds". The girl laughed at the Robot and said "The sum is definitely 648". "How can you tell that ?" Robot asked the girl. "Because I am an ACM World Finalist and I can solve the Big Number problems easily." Saying this, the girl closed her laptop computer and went away. Now, your task is to help the Robot with the similar problem.

**The Input**

The input file will contain one or more test cases. Each test case consists of one line containing an integer  $n$  ( $n \leq 1000$ ).

### The Output

For each test case, print one line containing the required number. This number will always fit into an integer, i.e. it will be less than  $2^{31}-1$ .

### Sample Input

```
5
60
100
```

### Sample Output

```
3
288
648
```

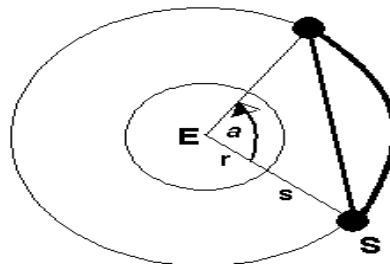
---

*Problem Setter : Ahmed Shamsul Arefin*

## Satellites

### The Problem

The radius of earth is 6440 Kilometer. There are many Satellites and Asteroids moving around the earth. If two Satellites create an angle with the center of earth, can you find out the *distance* between them? By *distance* we mean both the **arc** and **chord** distances. Both satellites are on the same orbit. (However, please consider that they are revolving on a circular path rather than an elliptical path.)



**E = Earth   S = Satellite**

### The Input

The input file will contain one or more test cases.

Each test case consists of one line containing two-integer  $s$  and  $a$  and a string "**min**" or "**deg**". Here  $s$  is the distance of the satellite from the surface of the earth and  $a$  is the angle that the satellites make with the center of earth. It may be in minutes ( ' ) or in degrees (  $^{\circ}$  ). Remember that the same line will never contain minute and degree at a time.

### The Output

For each test case, print one line containing the required distances i.e. both *arc distance* and *chord distance* respectively between two satellites in Kilometer. The distance will be a floating-point value with six digits after decimal point.

### Sample Input

```
500 30 deg
700 60 min
200 45 deg
```

### Sample Output

```
3633.775503 3592.408346
124.616509 124.614927
5215.043805 5082.035982
```

---

**Problem Setter :** *Ahmed Shamsul Arefin*

## Decode the Mad man

### The Problem

Once in BUET, an old professor had gone completely mad. He started talking with some peculiar words. Nobody could realize his speech and lectures. Finally the BUET authority fall in great trouble. There was no way left to keep that man working in university. Suddenly a student (definitely he was a registered author at UVA ACM Chapter and hold a good rank on 24 hour-Online Judge) created a program that was able to decode that

professor's speech. After his invention, everyone got comfort again and that old teacher started his everyday works as before.

So, if you ever visit BUET and see a teacher talking with a microphone, which is connected to a IBM computer equipped with a voice recognition software and students are taking their lecture from the computer screen, don't get thundered! Because now your job is to write the same program which can decode that mad teacher's speech!

### **The Input**

The input file will contain only one test case i.e. the encoded message.

The test case consists of one or more words.

### **The Output**

For the given test case, print a line containing the decoded words. However, it is not so hard task to replace each letter or punctuation symbol by the two immediately to its left alphabet on your standard keyboard.

### **Sample Input**

```
k[r dyt I[o
```

### **Sample Output**

```
how are you
```

---

***Problem Setter: Ahmed Shamsul Arefin***

## **How many nodes ?**

### **The Problem**

One of the most popular topic of Data Structures is Rooted Binary Tree. If you are given some nodes you can definitely able to make the maximum number of trees with them. But if you are given the maximum number of trees built upon a few nodes, Can you find out how many nodes built those trees?

**The Input**

The input file will contain one or more test cases. Each test case consists of an integer  $n$  ( $n \leq 4,294,967,295$ ). Here  $n$  is the maximum number of trees.

**The Output**

For each test case, print one line containing the actual number of nodes.

**Sample Input**

```
5
14
42
```

**Sample Output**

```
3
4
5
```

---

*Problem Setter: Ahmed Shamsul Arefin*

## Power of Cryptography

**Background**

Current work in cryptography involves (among other things) large prime numbers and computing powers of numbers modulo functions of these primes. Work in this area has resulted in the practical use of results from number theory and other branches of mathematics once considered to be of only theoretical interest.

This problem involves the efficient computation of integer roots of numbers.

**The Problem**

Given an integer  $n \geq 1$  and an integer  $p \geq 1$  you are to write a program that determines  $\sqrt[n]{p}$ , the positive  $n^{\text{th}}$  root of  $p$ . In this problem, given such integers  $n$  and  $p$ ,  $p$  will always be of the form  $k^n$  for an integer  $k$  (this integer is what your program must find).

**The Input**

The input consists of a sequence of integer pairs  $n$  and  $p$  with each integer on a line by itself. For all such pairs  $1 \leq n \leq 200$ ,  $1 \leq p < 10^{101}$ , and there exists an integer  $k$ ,  $1 \leq k \leq 10^9$ , such that  $k^n = p$ .

**The Output**

For each integer pair  $n$  and  $p$  the value  $\sqrt[n]{p}$  should be printed, i.e., the number  $k$  such that  $k^n = p$ .

**Sample Input**

```
2
16
3
27
7
4357186184021382204544
```

**Sample Output**

```
4
3
1234
```

## Roman Roulette

The historian Flavius Josephus relates how, in the Romano-Jewish conflict of 67 A.D., the Romans took the town of Jotapata which he was commanding. Escaping, Josephus found himself trapped in a cave with 40 companions. The Romans discovered his whereabouts and invited him to surrender, but his companions refused to allow him to do so. He therefore suggested that they kill each other, one by one, the order to be decided by lot. Tradition has it that the means for effecting the lot was to stand in a circle, and, beginning at some point, count round, every third person being killed in turn. The sole survivor of this process was Josephus, who then surrendered to the Romans. Which begs the question: had Josephus previously practised quietly with 41 stones in a dark corner, or had he calculated mathematically that he should adopt the 31st position in order to survive?

Having read an account of this gruesome event you become obsessed with the fear that you will find yourself in a similar situation at some time in the future. In order to prepare yourself for such an eventuality you decide to write a program to run on your hand-held PC which will determine the position that the counting process should start in order to ensure that you will be the sole survivor.

In particular, your program should be able to handle the following variation of the processes described by Josephus.  $n > 0$  people are initially arranged in a circle, facing inwards, and numbered from 1 to  $n$ . The numbering from 1 to  $n$  proceeds consecutively in a clockwise direction. Your allocated number is 1. Starting with person number  $i$ , counting starts in a clockwise direction, until we get to person number  $k$  ( $k > 0$ ), who is promptly killed. We then proceed to count a further  $k$  people in a clockwise direction, starting with the person immediately to the left of the victim. The person number  $k$  so selected has the job of burying the victim, and then returning to the position in the circle that the victim had previously occupied. Counting then proceeds from the person to his immediate left, with the  $k$ th person being killed, and so on, until only one person remains.

For example, when  $n = 5$ , and  $k = 2$ , and  $i = 1$ , the order of execution is 2, 5, 3, and 1. The survivor is 4.

### Input and Output

Your program must read input lines containing values for  $n$  and  $k$  (in that order), and for each input line output the number of the person with which the counting should begin in order to ensure that you are the sole survivor. For example, in the above case the safe starting position is 3. Input will be terminated by a line containing values of 0 for  $n$  and  $k$ .

Your program may assume a maximum of 100 people taking part in this event.

### Sample Input

```
1 1
1 5
0 0
```

### Sample Output

```
1
1
```

## Joseph's Cousin

### The Problem

The Joseph's problem is notoriously known. For those who are not familiar with the problem, among  $n$  people numbered  $1, 2, \dots, n$ , standing in circle every  $m$ th is going to be executed and only the life of the last remaining person will be saved. Joseph was smart enough to choose the position of the last remaining person, thus saving his life to give the message about the incident.

Although many good programmers have been saved since Joseph spread out this information, Joseph's Cousin introduced a new variant of the malignant game. This insane character is known for its barbarian ideas and wishes to clean up the world from silly programmers. We had to infiltrate some the agents of the ACM in order to know the process in this new mortal game. In order to save yourself from this evil practice, you must develop a tool capable of predicting which person will be saved.

### The Destructive Process

The persons are eliminated in a very peculiar order;  $m$  is a dynamical variable, which each time takes a different value corresponding to the prime numbers' succession (2, 3, 5, 7, ...). So in order to kill the  $i$ th person, Joseph's cousin counts up to the  $i$ th prime.

### The Input

It consists of separate lines containing  $n$  [1..3501], and finishes with a 0.

### The Output

The output will consist in separate lines containing the position of the person which life will be saved.

### Sample Input

```
6
0
```

### Sample Output

```
4
```

## Integer Inquiry

One of the first users of BIT's new supercomputer was Chip Diller. He extended his exploration of powers of 3 to go from 0 to 333 and he explored taking various sums of those numbers.

“This supercomputer is great,” remarked Chip. “I only wish Timothy were here to see these results.” (Chip moved to a new apartment, once one became available on the third floor of the Lemon Sky apartments on Third Street.)

### Input

The input will consist of at most 100 lines of text, each of which contains a single VeryLongInteger. Each VeryLongInteger will be 100 or fewer characters in length, and will only contain digits (no VeryLongInteger will be negative).

The final input line will contain a single zero on a line by itself.

### Output

Your program should output the sum of the VeryLongIntegers given in the input.

### Sample Input

```
123456789012345678901234567890
123456789012345678901234567890
123456789012345678901234567890
0
```

### Sample Output

```
370370367037037036703703703670
```

## The Decoder

Write a complete program that will correctly decode a set of characters into a valid message. Your program should read a given file of a simple coded set of characters and print the exact message that the characters contain. The code key for this simple coding is a one for one character substitution based upon a *single arithmetic manipulation* of the printable portion of the ASCII character set.

### Input and Output

For example: with the input file that contains:

```
1JKJ'pz'{ol'{yhlthyr'vm'{ol'Jvu{yvs'Kh{h'Jvywvyh{pvu5
1PIT'pz'h'{yhlthyr'vm'{ol'Pu{lyuh{pvuhs'I|zpulzz'Thjopul'Jvywvyh{pvu5
1KLJ'pz'{ol'{yhlthyr'vm'{ol'Kpnp{hs'Lx|pwtlu{'Jvywvyh{pvu5
```

your program should print the message:

\*CDC is the trademark of the Control Data Corporation.  
\*IBM is a trademark of the International Business Machine Corporation.  
\*DEC is the trademark of the Digital Equipment Corporation.

Your program should accept all sets of characters that use the same encoding scheme and should print the actual message of each set of characters.

### Sample Input

```
1JKJ'pz'{ol'{yhlthyr'vm'{ol'Jvu{yvs'Kh{h'Jvywvyh{pvu5
1PIT'pz'h'{yhlthyr'vm'{ol'Pu{lyuh{pvuhs'I|zpulzz'Thjopul'Jvywvyh{pvu5
1KLJ'pz'{ol'{yhlthyr'vm'{ol'Kpnp{hs'Lx|pwtlu{'Jvywvyh{pvu5
```

### Sample Output

\*CDC is the trademark of the Control Data Corporation.  
\*IBM is a trademark of the International Business Machine Corporation.  
\*DEC is the trademark of the Digital Equipment Corporation.

## **APPENDIX B**

### **COMMON CODES/ROUTINES FOR PROGRAMMING**

---



This part of this book contains some COMMON codes/routines/pseudocodes for programmers that one can EASILY use during the contest hours.<sup>[10]</sup>

**Finding GCD,LCM and Prime Factor**

```
#include<iostream>
#include<vector>
#include<fstream>
using namespace std;

int gcd(int a, int b)
{
    int r=1;
    if(a>b)
    {
        while ( r!=0)
        {
            r=a%b;
            a=b;
            b=r;
        }
    }
    return a;
}
else if ( b>a)
{
    while (r!=0)
    {
        r=b%a;
        b=a;
        a=r;
    }
}
return b;
}
else if ( a==b)
    return a;
}

int lcm(int a, int b)
{
    return a*b/gcd(a,b);
}

bool isprime(int a)
{
    {
        if(a==2) return true;
        else
        {
            for(int i=2;i<a/2+1;++i)
                if(a%i==0) return false;
            if(i==a/2) return true;}
    }
}
```

```
vector<int> primefactors(int a)
{
    vector<int> primefactors;
    for(int i=1;i<=a;++i)
        if(a%i==0&&isprime(i))
        {
            primefactors.push_back(i);
            a=a/i;i=1;
        }
    return primefactors;
}

int main()
{
    int a=5,b=2500;
    out<<"GCD = "<<gcd(a,b)<<endl;
    out<<"LCM = "<<lcm(a,b)<<endl;
    if(!isprime(b)) out<<"not
    prime"<<endl;
    vector <int> p =
    primefactors(b);
    for(int i=0;i<p.size();++i)
        out<<p[i]<<endl;
    return 0;
}
```

### Recursively find the GCD by Euclid's Algorithm

```

//*****      GCD using recursive way      *****/
#include<stdio.h>
unsigned int Euclid_g_c_d(unsigned int a, unsigned int b){
    if(b==0)
        return a;
    else
        return Euclid_g_c_d(b,a%b);
}
int main(){
    unsigned int a,b,gcd;
    while( scanf("%d%d",&a,&b)==2){
        gcd=Euclid_g_c_d(a,b);
        printf("%d",gcd);
    }
    return 0;
}

```

The another application of GCD is Extended Euclid's Algorithm..We can solve the formula  $d = ax + by \Rightarrow \text{gcd}(a,b) = ax + by$ . This algorithm take the inputs a and b..and out put the d(gcd of a,b) and the value of the root x and y.

```

//*****      Extended Euclid's Algorithm      *****/
#include<stdio.h>
unsigned long d,x,y;
void Extended_Euclid(unsigned long a, unsigned long b){
    unsigned long x1;
    if(b>a){
        x1=a;    //if b>a so I used this if condition
        a=b;    // result is ok but x and y swaped
        b=x1;
    }
    if(b==0){
        d=a;
        x=1;
        y=0;
        return;
    }
    Extended_Euclid(b,a%b);
    d = d;
    x1 = x-(a/b) * y;
    x = y;
    y = x1;
}
int main(){
    unsigned long a,b;    // d = gcd(a,b) = ax+by
    while(scanf("%lu %lu", &a, &b)==2){
        Extended_Euclid(a,b);
        printf("%lu %lu %lu\n",d,x,y);}return 0;}

```

**GCD of N numbers:**

This program takes a number that is for how many numbers you want to find the GCD and show the GCD of them. This takes input until end of file.

```
/******          GCD for n numbers          *****//
#include<stdio.h>
int main(){
    long gcd,a,b,n,i;
    while(scanf("%ld",&n)==1){
        scanf("%ld%ld",&a,&b);
        i=2;
        while(i<n){
            gcd=g_c_d(a,b);
            a=gcd;

i++;
            scanf("%ld",&b);
        }
        gcd=g_c_d(a,b);
        printf("%ld\n",gcd);
    }
    return 0;
}
```

**LCM (Lowest Common Multiple)**

It is also a important topic in programming. Suppose different clocks are set in 12 point, and each and every clock has their own time to complete the cycle. When every clock again set in the 12 point.? This type of problem can be solved by LCM. LCM was implemented by the help of GCD.

```
LCM (m, n) = [m / GCD(m, n)] * n;
Or,  D1=m / GCD(m, n);
      D2=n / GCD(m, n);
      LCM(m, n) = D1 * D2 * GCD(m, n);
```

Here is the code of some int numbers LCM.

```
long GCD(long a, long b){
    if(b==0)
        return a;
    else
        return GCD(b,a%b);}
long LCM(long a,long b){
```

```

        return (a/GCD(a,b)*b);
    }
int main(){
    long a,b;
    int n,i;
    while(scanf("%d",&n)==1){
        if(n==0) break;
        scanf("%ld",&a);
        for(i=1;i<n;i++){
            scanf("%ld",&b);
            a=LCM(a,b);
        }
        printf("%ld\n",a);
    }
    return 0;}

```

## Factorials

$n! = n * (n-1) * (n-2) * (n-3) \dots \text{upto } 1.$   
 $5! = 5 * (5-1) * (5-2) * (5-3) * 2 * 1 = 120.$   
 This is a recursion process.

```

/****          Factorial by recursion          *****/
#include<stdio.h>
long fac(long x){
    if(x==0)
        return 1;
    return x * fac(x-1);
}
/*****
int main(){
    long n,res;
    while(scanf("%ld",&n)==1){
        res=fac(n);
        printf("%ld\n",res);
    }
    return 0;}

```

This is very easy, but one thing we have to know that factorials increases multiply. The 12<sup>th</sup> factorial is 479001600 which is very big so can you imagine the factorial of 1000!....? The above program can compute upto 12<sup>th</sup> factorials so for less than 12<sup>th</sup> we can use it. But for time consideration in ACM we always try to avoid the recursion process.

### Iterative Factorial

```

//***** Factorial by normal multiplication *****//
#include<stdio.h>
long fac(long x){
    long i,res=1;
    for(i=1;i<=x;i++){
        res*=i;
    }
    return res;
}
int main(){
    long n,res;
    while(scanf("%ld",&n)==1){
        res=fac(n);
        printf("%ld\n",res);
    }
    return 0;
}

```

So, how can we compute the factorial of big numbers? Here is a code that can show the factorials up to 1000. And this is an ACM problem (623-500!). This code is very efficient too!. We pre-calculate all the 1000<sup>th</sup> factorials in a two dimensional array. In the main function just takes input and then just print.

### Factorial of Big Number!

```

#include<stdio.h>
#include<string.h>
char f[10000];
char factorial[1010][10000];
void multiply(int k){
    int cin,sum,i;
    int len = strlen(f);
    cin=0;
    i=0;
    while(i<len){
        sum=cin+(f[i] - '0') * k;
        f[i] = (sum % 10) + '0';
        i++;
        cin = sum/10;
    }
    while(cin>0){
        f[i++] = (cin%10) + '0';
        cin/=10;
    }
    f[i]='\0';
    for(int j=0;j<i;j++){
        factorial[k][j]=f[j];
    }
    factorial[k][i]='\0';
}

```

```
void fac(){
    int k;
    strcpy(f,"1");
    for(k=2;k<=1000;k++)
        multiply(k);
}
void print(int n){
    int i;
    int len = strlen(factorial[n]);
    printf("%d!\n",n);
    for(i=len-1;i>=0;i--){
        printf("%c",factorial[n][i]);
    }
    printf("\n");
}
int main(){
    int n;
    factorial[0][0]='1';
    factorial[1][0]='1';
    fac();
    while(scanf("%d",&n)==1){
        print(n);
    }
    return 0;
}
```

## Factorial Frequencies

Now how many digits in the factorial of N numbers..? Yes we can count them by the above process. But we can do it simply!

```
/****** How many digits in N factorials *****//
#include<stdio.h>
#include<math.h>
/*****/
double count_digit(long a){
    double sum;
    long i;
    if(a==0) return 1;
    else{
        sum=0;
        for(i=1;i<=a;i++){
            sum+=log10(i);
        }
        return floor(sum)+1;
    }
}
/*****/
int main(){
    double sum;
    long n;
```

```

while (scanf ("%ld", &n) == 1) {
    sum = count_digit (n);
    printf ("%01f\n", sum);
}
return 0;
}

```

### How many trailing zeros in Factorials ?

We know that factorials of any number has so many zeros (0's) at last...example  $17! = 355687428096000$ . It has 3 last zero digits. So we now see how many trailing zeros have in  $N$  factorials.

```

//*****          How many trailing zeros in N!
*****/*****//
#include<stdio.h>
long zero(long number, long factor) {
    long total, deno;
    if (number == 5) return 1;
    total = 0;
    deno = factor;
    while (deno < number) {
        total += number / deno;
        deno *= factor;
    }
    return total;
}
/*****/
int main() {
    long N, c2, c1;
    while (scanf ("%ld", &N) == 1) {
        c1 = zero (N, 2);
        c2 = zero (N, 5);
        if (c1 < c2) printf ("%ld\n", c1);
        else printf ("%ld\n", c2);
    }
    return 0;
}

```

### Big Mod

We are not always interested in the full answers, however. Sometimes the remainder suffices for our purposes. In that case we use Modular Arithmetic. The key to efficient modular arithmetic is understanding how the basic operation of addition, subtraction, and multiplication work over a given modulus.

- Addition:-  $(x + y) \bmod n \dots ? = ((x \bmod n) + (y \bmod n)) \bmod n$ .
- Subtraction:-  $(x - y) \bmod n \dots ? = ((x \bmod n) - (y \bmod n)) \bmod n$ .

We can convert a negative number mod  $n$  to positive number by adding a multiple of  $n$  to it.

➤ Multiplication:-  $xy \bmod n \dots? = (x \bmod n)(y \bmod n) \bmod n$ .

**What is  $x^y \bmod n \dots?$**  Exponentiation is just repeated multiplication,  $(x \bmod n)^y \bmod n$ . So if the  $x$  and  $y$  are very big equal to 32 bit. Then we can also compute it without overflow.

```
/* Big mod */
#include<stdio.h>
long square(long s){
    return s*s;
}
long bigmod(long b, long p, long m){
    if (p == 0)
        return 1;
    else if (p%2 == 0)
        return square( bigmod (b,p/2,m)) % m; // square(x) = x * x
    else
        return ((b % m) * bigmod( b,p-1,m)) % m;
}
int main(){
    long b,p,m,sum;
    while(scanf("%ld%ld%ld",&b,&p,&m)==3){
        sum = bigmod( b, p, m);
        printf("%ld\n", sum);
    }
    return 0;}
```

## Number Conversions - Integer ⇔ Binary

### Convert an integer number into binary number

```
int inttobin(unsigned int a,int
*bin)
{
    int one=0;
    unsigned int c=1;
    int i ;
    for(i=31;i>=0;i--)
    {
        one = one + (a&c);
        bin[i] = (a&c)?1:0;
        c<<=1;
    }
    return one; }
```

### Convert a binary number into binary number

```
unsigned int creatnum(int *num)
{
    int i;
    unsigned int a=0;
    for(i=0;i<32;i++)
    {
        a= a|num[i];
        if(i!=31)
            continue;
        a <<= 1;
    }
    return a;
}
```

**Integer ⇔ hex ⇔ Integer**

```
char hex[100];
int num = 4095;
sprintf(hex,"%X",num); // convert the num in upper case hex decimal in hex
sprintf(hex,"%x",num); // convert the num in lower case hex decimal in hex
sscanf(hex,"%x",&num); // convert the hex number hex in integer num
```

**Integer to any base**

Following code help you to convert any integer number on any base from 2 to 36.

```
#include<string.h>
#include<stdio.h>
#define MAXDGT 200          /* maximum number of digit */

/* ***** */
/*      A general swap function that swap two object.      */
/*      Input   : Two object.                                */
/*      Return  : None                                       */
/* ***** */
template <class T>

void swap(T &x, T &y)
{   T tmp=x;   x=y;   y=tmp;   };

/* ***** */
/*      A general string reverse function that reverse the  */
/*      passed string and store the string int to the parameter. */
/*      Input   : A string                                    */
/*      Return  : None                                       */
/* ***** */
void revstr(char *str)
{   int i=0,l=strlen(str);
    while(i<l)
        swap(str[i++],str[--l]);
}

/* ***** */
/*      A general base conversion function                  */
/*      Input   : A number n and a base b                   */
/*      Return  : A character array which contain the number n in base b */
/* ***** */
char *itob(long int n,int b=10)
{   char num[MAXDGT];
    int j,sign;
    register int i=0;
```

```

        if( (sign=n) <0 )
            n= -n;
do    {      j=n%b;
        num[i++]=(j<10) ? (j+'0') : ('A'+j-10);
        }while((n/=b) !=0);

    if(sign < 0)        num[i++]='-';

    num[i]='\0';
    revstr(num);
    return num;
}

/* Sample main */
main(void)
{    printf(itob(71],36));    }

```

## Decimal To Roman

Roman number is very useful number system. Often we need to convert a decimal number into Roman number. In Roman number the following symbol are used.

i, v, x, l, c, m for 1, 5, 10, 50, 100 and 500 respectively.

```
#include<stdio.h>
```

```

/* ***** */
/*      Convert number 1 to 10      */
/*      Input   : A integer number   */
/*      Return  : None                */
/* ***** */

void unit(int n)
{
    switch(n){
        case 3 : printf("i");
        case 2 : printf("i");
        case 1 : printf("i"); break;
        case 4 : printf("i");
        case 5 : printf("v"); break;
        case 6 : printf("vi"); break;
        case 7 : printf("vii"); break;
        case 8 : printf("viii"); break;
        case 9 : printf("ix"); break;
    }
}

/* ***** */
/*      Convert number 10 to 100     */
/* ***** */

```

```

/*      Input   : A integer number          */
/*      *****                               */
void ten(int n)
{
    switch(n) {
        case 3 : printf("x");
        case 2 : printf("x");
        case 1 : printf("x"); break;
        case 4 : printf("x");
        case 5 : printf("l"); break;
        case 6 : printf("lx"); break;
        case 7 : printf("lxx"); break;
        case 8 : printf("lxxx"); break;
        case 9 : printf("xc"); break;
    }
}

/*      *****                               */
/*      Convert number 100 to 500             */
/*      Input   : A integer number          */
/*      *****                               */

void hnd(int n)
{
    switch(n) {
        case 3 : printf("c");
        case 2 : printf("c");
        case 1 : printf("c"); break;
        case 4 : printf("c");
        case 5 : printf("M"); break;
    }
}

/*      *****                               */
/*      Convert an integer number into roman system */
/*      Input   : A integer number          */
/*      *****                               */
void roman(int n)
{
    int a,i;
    if(n>=500)
    {
        a=n/500 ;
        for(i=1;i<=a;i++)
            printf("M");
    }
    n=n%500;
    hnd(n/100);
    n=n%100;
    ten(n/10);
    unit(n%10);
}

main(void)
{
    roman(390);
    return 0;
}

```

## Josephus Problem

Flavius Josephus was a famous historian of the first century. Legend has it that Josephus wouldn't have lived to become famous without his mathematical talents. During the Jewish-Roman war, he was among a band of 41 Jewish rebels trapped in cave by the Romans. Preferring suicide to capture, the rebels decided to form a circle and, proceeding around it, to kill every third remaining person until no one was left. But Josephus, along with an un-indicted co-conspirator, wanted none of this suicide nonsense; so he quickly calculated where he and his friend stand in the vicious circle.

```
#include <stdio.h>
#define MAX 150

/** Implementation of circular queue **/

int queue[MAX];
int e, f, r;

/** _f is front; _r is rear & _e is element number **/

/* ***** */
/* A general push algorithm act over queue */
/* ***** */

void push(int i)
{
    r++;
    e++;
    if (r >= MAX)
        r = 1;
    queue[r] = i;
}

/* ***** */
/* A general pop algorithm act over queue */
/* ***** */

int pop(void)
{
    e--;
    int i = queue[f];
    f++;
    if (f >= MAX)
        f = 1;
    return i;
}
```

```
/** End of circular queue **/  
/* ***** */  
/* A general joshuf function. */  
/* This function start removing */  
/* element from first element */  
/* and return the sirviver */  
/* ***** */  
  
int joshups(int n,int v)  
{  
    register int i;  
    e=n;  
    for(i=1;i<=n;i++)  
        queue[i] = i;  
    f = 1; // 0 for serviving first element.  
    r = n; // n+1 for serviving first element.  
    i = 0;  
    if(n > 1)  
        pop();  
    while(e!=1)  
    {  
        if(i!=v)  
        {  
            i++;  
            push(pop());  
        }  
        else  
        {  
            pop();  
            i = 0;  
        }  
    }  
    return queue[f];  
}  
  
/*sample main function*/  
  
main(void)  
{  
    int i,m;  
    scanf("%d",&i);  
    while(i)  
    {  
        m=1;  
        while((joshups(i,m++)) != 13 );  
        printf("%d",m);  
        putchar('\n');  
        scanf("%d",&i);  
    }  
  
    return 0; }
```

## Combinations

```

/* ***** */
/*      Calculate nCm      */
/*      Input : Two integer number n m      */
/*      Return : The nCm      */
/* ***** */

double nCr(int n,int m)
{
    int k;
    register int i,j;
    double c,d;

    c=d=1;
    k=(m>(n-m)) ? m: (n-m);
    for(j=1,i=k+1; i<=n; i++,j++)
    {
        c*=i;
        d*=j;
        if( !fmod(c,d)  && (d!=1) )
        {   c/=d;
            d=1;
        }
    }

    return c;
}

/* A sample main function */
main(void)
{
    int n,m;

    while( scanf("%d%d",&n,&m) !=EOF)
        printf("%.0lf\n",nCr(n,m));
    return 0;
}

```

**Another way to calculate the  $nC_m$  by using the Pascal's triangel.**

```

#include<stdio.h>
#define MAXTRI 50

unsigned long int pas[MAXTRI][MAXTRI];

void pascals(int n)
{

```